
The Piggyback Bitmap

Complete reference for the confirmation mechanism — every case explained with the logic of how and why it is handled

What it is · how confirmations are recorded, windowed, and sent · how the sender consumes them
new-site join · reconnect · overflow · every failure case it defends against

Scope: the piggyback subsystem only (receiver-side accumulation in `struct piggy_pending`, sender-side `confirmed_range` list, the PIGGY TLV, and every guard around them). Other modules appear only where they touch piggyback.

Contents

1. What the piggyback bitmap is and why it exists
2. The two structures and every field
3. The PIGGY TLV on the wire
4. Receiver side: recording a confirmation
5. Receiver side: flushing (threshold, timer, draining)
6. Sender side: consuming a piggyback
7. The guards that make it safe
8. New site joins
9. Worked numeric example
0. Every case, explained with the logic
1. Known issues & tuning

1 What the piggyback bitmap is and why it exists

The accelerator saves bandwidth by sending a 32-byte hash (**HIT**) instead of a full 4 KB chunk (**MISS**) whenever the receiver already holds that chunk. A HIT is only safe if the sender *knows for certain* the receiver has the chunk durably on disk. The **piggyback bitmap** is the receiver's message back to the sender that says, compactly, "here are the exact chunk ids I have stored."

One sentence: the piggyback bitmap is a 64-bit field plus a base id, sent receiver→sender, where bit N set means "I have durably stored chunk `base + N`." The sender turns those bits into confirmed ranges and only sends a HIT for a chunk inside a confirmed range.

1.1 Why a bitmap and not a single watermark

The earlier design used one number — "I have everything up to N." That cannot express holes. If chunk 5 is stored but chunk 4 is still in flight, a watermark must either stall at 3 (losing the confirmation for 5) or advance past 4 (falsely claiming a chunk never stored). The bitmap records each id independently, so gaps are represented exactly.

Confirmed	Watermark says	Bitmap says
1,2,3,5,6 (4 missing)	3 (5 and 6 lost)	base=1, bits 0,1,2,4,5 — exact
later 4 arrives	can jump to 6	bit 3 set → runs merge to 1..6

1.2 The two halves of the mechanism

- **Receiver side** accumulates confirmations into a 64-bit window (`struct piggy_pending`) and periodically serialises it into a PIGGY TLV.
- **Sender side** parses each PIGGY TLV and folds the set bits into a list of merged `[lo,hi]` ranges (`struct confirmed_range`), which it queries before every HIT.

2 The two structures and every field

2.1 Receiver: `struct piggy_pending`

Field	Type	Meaning
<code>base</code>	<code>uint32_t</code>	Lowest <code>sender_cid</code> the current window covers; corresponds to bit 0.
<code>bitmap</code>	<code>uint64_t</code>	64 confirmation bits; bit N set ⇒ <code>cid base+N</code> is durably stored.
<code>count</code>	<code>uint32_t</code>	popcount(bitmap), kept incrementally so the batch threshold needs no rescan.
<code>timer</code>	<code>uint32_t</code>	Countdown ticks to a forced flush; armed when the first bit in a window is set.
<code>ready</code>	<code>int</code>	1 ⇒ the window must be sent at the next opportunity.
<code>has_queued</code>	<code>int</code>	1 ⇒ an older, full window is saved and must be sent before the current one.
<code>queued_base</code>	<code>uint32_t</code>	Base of the saved older window.
<code>queued_bitmap</code>	<code>uint64_t</code>	Bitmap of the saved older window.

2.2 Sender: `struct confirmed_range` + the list

Field	Type	Meaning
<code>lo</code>	<code>uint32_t</code>	Inclusive lower bound of a confirmed run of <code>sender_cids</code> .
<code>hi</code>	<code>uint32_t</code>	Inclusive upper bound.
<code>ranges[16]</code>	array	Up to <code>MAX_CONFIRMED_RANGES</code> sorted, merged intervals (in <code>module1_state</code>).
<code>nranges</code>	<code>int</code>	Number of active entries in <code>ranges</code> .
<code>store_verified</code>	<code>int</code>	0 until the first valid piggyback arrives; gates every HIT on this connection.
<code>conn_epoch</code>	<code>uint32_t</code>	This connection's epoch; stamped into outgoing PIGGY and checked on incoming.

2.3 The constants that govern it

Constant	Value	Role
<code>PIGGY_BITMAP_BITS</code>	64	Width of one window; offset ≥ this triggers rollover.
<code>PIGGY_BATCH_THRESHOLD</code>	32	Confirm count that flushes immediately.
<code>PIGGY_TIMER_TICKS</code>	5	Ticks before a forced flush of pending confirmations.
<code>MAX_CONFIRMED_RANGES</code>	16	Max non-contiguous confirmed regions the sender tracks.
<code>PIGGY_PAYLOAD_SIZE</code>	14 (should be 18)	Serialised payload length — see §11.

3 The PIGGY TLV on the wire

A PIGGY TLV is an ordinary TLV: a 3-byte header (type + 16-bit length) followed by the payload. It is identified purely by its *type byte* **0xFC** — the decoder never scans data looking for magic bytes, which is why magic bytes occurring by chance inside chunk data can never be mistaken for a piggyback.

```
Header:
  byte 0 : 0xFC          type = PIGGY
  byte 1 : val_len high
  byte 2 : val_len low    val_len = 18

Payload (18 bytes, big-endian):
  [0xAB][0xCD]          magic          2 B
  [epoch  4 B]          reconnect guard 4 B
  [base   4 B]          lowest cid (bit 0) 4 B
  [bitmap 8 B]          64 confirmation bits 8 B

Full PIGGY TLV on the wire = 3 + 18 = 21 bytes.
```

Each PIGGY TLV is fully self-describing: it carries its own epoch, base, and bitmap. That is why two piggybacks arriving out of order are handled correctly — each is processed independently and the sender's range list absorbs both.

4 Receiver side: recording a confirmation

Whenever a chunk is durably stored (the verified-store call has flushed it to disk), the receiver calls `piggy_record_confirmed(s, sender_cid)` to set that chunk's bit. The *sender's* cid is used — carried in the MISS TLV's 4-byte prefix — so the two sides agree on which bit means which chunk.

`piggy_record_confirmed(s, sender_cid)` – step by step

- 1 **Sentinel guard:** if `sender_cid == 0`, return immediately. 0 is the universal “invalid” value and must never set a bit.
- 2 **First confirmation:** if `base == 0` (window unused), adopt this cid as the `base` — it becomes bit 0.
- 3 Compute `offset = sender_cid - base`.
- 4 **Overflow path** (`offset ≥ 64`): the cid does not fit the current window.
 - If the current window holds confirmations (`count > 0`):
 - If nothing is queued yet, save the current `base + bitmap` into the queued slot and set `has_queued = 1`.
 - If a window is *already* queued, set `ready = 1` to force an immediate flush — there is only one queue slot.
 - Open a fresh window: `base = sender_cid`, `bitmap = 0`, `count = 0`, `offset = 0`.
- 5 **Duplicate guard:** if the bit at `offset` is already set, return without changing `count` — a repeated confirmation must not inflate the count.
- 6 Set the bit (`bitmap |= 1ULL << offset`) and increment `count`.
- 7 **Arm the timer** if it is not already running (`timer = PIGGY_TIMER_TICKS`).
- 8 **Threshold check:** if `count ≥ PIGGY_BATCH_THRESHOLD` (32), set `ready = 1`.

5 Receiver side: flushing the bitmap

A pending window leaves the receiver as a PIGGY TLV when any of three things happens.

Trigger	Where	Condition	Effect
Batch threshold	<code>piggy_record_confirmed</code>	<code>count ≥ 32</code>	<code>ready=1</code> ; sent next opportunity
Timer	<code>byte_caching_timer_check</code>	5 ticks elapsed with anything pending	standalone PIGGY TLV sent on the reverse socket
Overflow	<code>piggy_record_confirmed</code>	<code>offset ≥ 64</code>	queue old window, open new

The timer path (`byte_caching_timer_check`)

- 1 If the pending timer is not armed (`timer == 0`), do nothing.
- 2 Decrement the timer. When it reaches 0, if anything is pending (`count > 0` or `has_queued`), set `ready = 1`.
- 3 If a reverse socket (`so_receiver`) is set, send a standalone PIGGY TLV immediately.
- 4 If no socket is set, leave `ready = 1` so the next outgoing data TLV carries the piggyback.

The timer is what guarantees a **small transfer** (fewer than 32 chunks) and a **receive-only session** (no reverse data traffic) still send their confirmations. Without it, a window that never reaches 32 and never rides on reverse traffic would sit unsent forever.

Draining a window (`module1_get_pending_piggy`)

When a PIGGY TLV is built, this function hands back exactly one window and clears it.

- 1 **Empty guard:** if `count == 0` and nothing is queued, clear `ready` and return 0 — no TLV is built, so an empty bitmap is never sent.
 - 2 Output the current `conn_epoch`.
 - 3 **Queued first:** if a window is queued, output it (the older one) and clear the queue. If the current window still has data, keep `ready = 1` so the next call sends it too.
 - 4 Otherwise output the current window's `base / bitmap` and fully reset the current window (base, bitmap, count, ready, timer all to 0).
- Draining the queued window first preserves ordering and guarantees no confirmed window is ever lost, even under overflow.

6 Sender side: consuming a piggyback

When a PIGGY TLV arrives, the sender parses it and folds the bits into its range list.

`piggy_process_incoming(s, payload)` – step by step

- 1 **Magic check:** if bytes 0–1 are not `0xAB 0xCD`, discard (corruption or a partial trailer).
- 2 Decode big-endian `epoch` (bytes 2–5), `base` (6–9), and the 64-bit `bitmap` (10–17).
- 3 **Epoch guard:** if `epoch != conn_epoch`, this piggyback is from a previous connection — discard.
- 4 **Base guard:** if `base == 0`, discard (invalid sentinel).
- 5 **Run extraction:** walk bits 0...63. Each maximal run of consecutive set bits becomes a range `[base+first, base+last]`, passed to `piggy_insert_range`. Holes stay holes.
- 6 **Mark verified:** on the first valid piggyback, set `store_verified = 1`.

`piggy_insert_range(s, lo, hi)` – merge logic

- 1 Reject if `lo == 0`, `hi == 0`, or `lo > hi`.
- 2 **Merge:** if `[lo,hi]` overlaps or is adjacent to an existing range (`lo ≤ r[i].hi+1` and `hi ≥ r[i].lo-1`), widen that range to include it, then absorb any following ranges it now touches (shifting the array down). Done.
- 3 **Full-list guard:** if no range touched and the list is at 16 entries, log a warning and drop the new range — existing entries are never corrupted. The sender keeps sending MISS for the dropped region (safe, just un-deduped).
- 4 Otherwise insert at the sorted position, shifting entries up.

`piggy_is_confirmed(s, cid)` – the HIT gate

Returns 1 only if **all three** hold: `store_verified == 1`, `cid != 0`, and `cid` lies inside some range `[lo,hi]`. This exact membership test is what replaced the watermark comparison — it never claims a cid sitting in a gap.

7 The guards that make it safe

Guard	Where	Protects against
<code>sender_cid == 0</code> ignored	<code>record_confirmed</code>	The invalid sentinel setting a bogus bit.
duplicate bit skipped	<code>record_confirmed</code>	Re-confirmation inflating <code>count</code> and false-triggering the threshold.
one queued window	<code>record_confirmed</code>	Losing an old window on overflow (it is saved, not discarded).
empty-bitmap return 0	<code>get_pending_piggy</code>	Sending a useless zero-confirmation TLV.
magic check	<code>process_incoming</code>	Acting on a corrupt or partial payload.
epoch check	<code>process_incoming</code>	Trusting a stale piggyback after reconnect.
<code>base == 0</code> reject	<code>process_incoming</code>	A malformed window anchored at the sentinel.
run extraction (not watermark)	<code>process_incoming</code>	Confirming chunks inside gaps.
full-list warn + drop	<code>insert_range</code>	Corrupting the range list when it overflows.
<code>store_verified</code> gate	<code>is_confirmed</code>	Sending any HIT before the receiver has proven it stored anything.
epoch bump on init	<code>state_init</code>	Old-connection piggybacks being accepted on a new connection.

8 New site joins

When a new site connects, the per-connection piggyback state is wiped by `module1_state_init`: `store_verified = 0`, empty range list, fresh `conn_epoch`, empty pending window. The shared disk store may already hold data, but *this connection has confirmed nothing*.

8.1 A matching hash on a fresh connection

Because `piggy_is_confirmed` requires `store_verified == 1`, the very first time a chunk is referenced on a new connection it cannot HIT — even if the data is physically on disk.

Hash in store	store_verified	cid in range	Sender sends
yes	0	no	MISS (warm-up downgrade)
yes	1	yes	HIT

8.2 How trust is earned

- 1 Sender sends the chunk as MISS (warm-up).
- 2 Receiver stores it (or finds it already present and bumps the ref), then `piggy_record_confirmed(sender_cid)` sets the bit.
- 3 On threshold or timer, the receiver sends a PIGGY TLV.
- 4 Sender's `piggy_process_incoming` validates epoch + base, inserts the range, and sets `store_verified = 1`.
- 5 From now on, matching hashes inside a confirmed range become HITS.

Warm-up is intentional. A match in the shared store is never enough on its own; it must be a match *plus* an epoch-matching confirmation on this connection. This is what protects a new or reconnected site from receiving a HIT it cannot satisfy. Every new site re-earns trust independently — two connections never share confirmation state.

9 Worked numeric example

Three bursts confirm in order: **21-40**, then **0-30**, then **100-200**.

Burst 1 — 21-40

```
base=21; bits 0..19 set; bitmap=0x000000000000FFFFF; count=20 (<32 -> timer flush)
Sender: one run -> range list = [21,40]
```

Burst 2 — 0-30 (overlaps below)

```
cid 0 ignored (sentinel). base=1; bits 0..29 -> ids 1..30; count=30 (timer flush)
Sender receives [1,30]; overlaps [21,40] -> MERGE -> range list = [1,40]
```

A watermark sat at 40 cannot record 1-30 below it without loss. The range list merges cleanly to [1,40], and cid 0 is correctly absent.

Burst 3 — 100-200 (exceeds one 64-bit window)

```
base=100; ids 100..163 fill bits 0..63 (full); id 164 overflows:
  queue window (base=100); open new window base=164; ids 164..200 fill bits 0..36
Drain queued FIRST -> TLV[base=100] run [100,163]; then TLV[base=164] run [164,200]
164 adjacent to 163 -> MERGE -> [100,200]
Final range list = [1,40] [100,200]
```

Resulting HIT/MISS decisions

Chunk	In range?	Decision
25	[1,40]	HIT
150	[100,200]	HIT
60	gap 41-99	MISS
0	sentinel	MISS

10 Every case, explained with the logic

Each case below follows the same four-part structure so the *reasoning* is explicit, not just the outcome:

- **Problem** — the situation that arises.
- **Failure if unhandled** — what would go wrong with naive code.
- **Mechanism** — the exact code construct that handles it.
- **Why sufficient** — why that construct fully closes the hole.

Legend: **FIXED** defends against a failure · **BY DESIGN** intentional safe behaviour · **PARTIAL** handled with a noted limit.

Case 1 — Confirmations arrive with gaps **FIXED**

Problem. Chunks 1,2,3,5,6 are stored but 4 is still in flight, so confirmations are non-contiguous.

Failure if unhandled. A single watermark would have to stop at 3 — it cannot say “up to 6 except 4” — so the confirmations for 5 and 6 are thrown away and those chunks can never become HITs.

Mechanism. Each id sets an independent bit (`bitmap |= 1ULL<<offset`). On the sender, `piggy_process_incoming` walks the bits and emits a range only for each *maximal run of set bits*, so it produces [1,3] and [5,6] and skips bit 3.

Why sufficient. The representation has one bit per id, so any subset of 64 ids is expressible exactly. There is no lossy projection onto a single number, so no confirmation is discarded. When 4 later arrives, its run is adjacent to both neighbours and `piggy_insert_range` merges all three into [1,6].

Case 2 — Sparse, far-apart confirmed regions **FIXED**

Problem. The confirmed ids are two clusters far apart, e.g. [1,40] and [100,200], with a genuine 60-id hole between them.

Failure if unhandled. A watermark collapses everything to one boundary; it must either falsely include 41-99 or exclude 100-200. Either way the sender makes wrong HIT/MISS choices in the gap.

Mechanism. The sender stores a *list* of up to `MAX_CONFIRMED_RANGES` (16) intervals, kept sorted and merged. The gap is simply the absence of a range covering 41-99.

Why sufficient. A list of disjoint intervals is the natural exact representation of an arbitrary set of contiguous runs. `piggy_is_confirmed` tests literal membership (`cid>lo && cid<hi`) across the list, so a cid in the hole matches no interval and correctly yields MISS.

Case 3 — A confirmation for cid 0 **FIXED**

Problem. 0 is the system-wide “invalid / not found” value (`find_by_hash` and `store_*` all return 0 on failure), yet a 0 could be passed into the confirmation path.

Failure if unhandled. If 0 became the window `base`, every real id would be offset from a meaningless anchor, and a spurious bit-0 confirmation would claim a non-existent chunk.

Mechanism. `piggy_record_confirmed` returns immediately when `sender_cid == 0`; symmetrically the sender rejects an incoming payload whose `base == 0`.

Why sufficient. The sentinel is filtered at both the entry (receiver records) and exit (sender parses) of the piggyback path, so a 0 can neither set a bit nor anchor a window. The guard is a single equality check that cannot be bypassed by any later code.

Case 4 — The same cid confirmed twice **FIXED**

Problem. A retransmit or duplicate delivery causes the same chunk to be stored-and- confirmed more than once.

Failure if unhandled. `count` would be incremented twice for one id, so the batch threshold (32) could trip after fewer than 32 *distinct* confirmations — sending a window that claims more than it holds, and corrupting the popcount invariant.

Mechanism. Before setting the bit, the code tests `if (bitmap & (1ULL<<offset)) return;` — if the bit is already set it returns without touching `count`.

Why sufficient. Idempotence is enforced at the bit level: setting an already-set bit is a no-op, and the early return guarantees `count` stays equal to the true popcount. So `count` can only ever reflect distinct confirmed ids.

Case 5 — A transfer too small to reach the batch threshold **FIXED**

Problem. Only 5 chunks are sent (96-100), so `count` never reaches the threshold of 32.

Failure if unhandled. If a window were sent only on reaching 32, these 5 confirmations would sit forever, the sender would never learn the receiver has them, and identical chunks would always be re-sent as MISS — defeating dedup for small flows.

Mechanism. The first confirmation in a window arms a countdown (`timer = PIGGY_TIMER_TICKS`, 5). `byte_caching_timer_check` decrements it each tick and, on reaching 0, sets `ready=1` if *anything* is pending and sends the window.

Why sufficient. The threshold and the timer are two independent triggers; the timer has no minimum count, so even a single pending confirmation is guaranteed to be flushed within 5 ticks regardless of how far below the threshold it is.

Case 6 — A receive-only session with no reverse traffic FIXED

Problem. The piggyback is conceptually “piggybacked” on reverse data TLVs, but a pure download has no reverse data to attach to.

Failure if unhandled. With nothing to ride on, the bitmap would never leave the receiver, so dedup could never turn on for a one-directional transfer.

Mechanism. When the timer fires, if a reverse socket `so_receiver` is set, `module4_send_standalone_piggy` builds and sends a PIGGY TLV *by itself*. If no socket is set, `ready=1` is held so the next outgoing TLV (whenever one occurs) carries it.

Why sufficient. The standalone path removes the dependency on reverse data entirely: the PIGGY TLV is a complete, self-contained packet, so confirmation does not require any other traffic to exist.

Case 7 — A confirmation falls outside the 64-bit window FIXED

Problem. A window covers exactly 64 consecutive ids from `base`. Confirming an id with `offset ≥ 64` (e.g. `base=96` then cid 200, or filling 100–163 then 164) does not fit.

Failure if unhandled. Naively re-basing the window to the new id would silently discard all the confirmations already accumulated in the old window.

Mechanism. On overflow the code first saves the current `base + bitmap` into the queued slot (`has_queued=1`), then opens a new window at the out-of-range id. `module1_get_pending_piggy` drains the queued (older) window *first*, then the current one.

Why sufficient. The old window is preserved verbatim rather than overwritten, and draining it first preserves chronological order. Both windows reach the sender as separate self-describing PIGGY TLVs, and the range list merges them if they happen to abut (164 next to 163 → one range).

Case 8 — A second overflow before the queue is drained PARTIAL

Problem. Confirmations are so scattered that a third window is needed while one window is already queued and the current one is in use.

Failure if unhandled. With only one queue slot, blindly overwriting the queued window would lose it; ignoring the new id would lose that.

Mechanism. When `has_queued` is already 1, the code does not attempt a second queue; instead it sets `ready=1`, forcing an immediate flush of what exists before opening the new window.

Why sufficient (with a limit). Nothing is lost — the forced flush pushes the backlog out before the new window takes over. The limit is throughput, not correctness: heavily out-of-order confirmation across >128 ids causes more frequent flushes (more PIGGY TLVs). A deeper queue would reduce that overhead.

Case 9 — The sender’s range list fills up FIXED (logged)

Problem. More than `MAX_CONFIRMED_RANGES` (16) disjoint regions are confirmed at once.

Failure if unhandled. Writing a 17th entry would overrun the fixed array and corrupt adjacent state, or an unchecked insert could overwrite a valid range.

Mechanism. `piggy_insert_range` first tries to merge into an existing range; only a genuinely new region needs a new slot. If the list is full it logs a warning and *drops* the new range rather than touching existing entries.

Why sufficient. Dropping a confirmation is always safe because the fallback is MISS: the sender simply keeps re-sending that region’s chunks in full. Correctness is preserved (no data lost, no array overrun); only dedup efficiency dips, and the log tells the operator to raise the constant. In practice merging keeps the live count well under 16.

Case 10 — Incoming payload anchored at base 0 FIXED

Problem. A malformed or corrupted PIGGY TLV arrives with `base == 0`.

Failure if unhandled. Every bit would map to `0 + offset`, inserting ranges starting at low, likely-bogus ids and possibly colliding with real chunk ids.

Mechanism. `piggy_process_incoming` rejects the payload outright if `base == 0`, before any range insertion.

Why sufficient. Since the receiver never *produces* a base of 0 (Case 3), a base of 0 on the wire can only be corruption, so rejecting it loses nothing legitimate while blocking the bogus insert.

Case 11 — A stale piggyback after reconnect **FIXED**

Problem. A connection drops and a new one is established, but a PIGGY TLV from the old connection is still in flight and arrives on the new one.

Failure if unhandled. The old confirmations refer to a now-irrelevant state; applying them could mark chunks confirmed that the new connection has not actually established, leading to unsafe HITs.

Mechanism. `module1_state_init` assigns each connection a unique `conn_epoch ( ++g_epoch_counter )`. Every PIGGY TLV carries the epoch it was built under, and `piggy_process_incoming` discards any payload whose epoch does not match the current `conn_epoch`.

Why sufficient. The epoch is a monotonically increasing tag that uniquely identifies the connection generation. A stale piggyback necessarily carries an older epoch, so the equality check rejects it deterministically — there is no window in which old and new epochs collide.

Case 12 — A corrupt or partial payload **FIXED**

Problem. Bytes are corrupted in transit, or the connection drops partway through a PIGGY TLV.

Failure if unhandled. Parsing garbage as epoch/base/bitmap could insert wild ranges or flip `store_verified` on bad data.

Mechanism. Two layers. (a) The decoder only calls the handler once it has collected the full declared value length, so a truncated TLV is simply never processed. (b) `piggy_process_incoming` verifies the leading magic bytes `0xAB 0xCD` and discards the payload on mismatch.

Why sufficient. Length-completeness handles truncation; the magic check is a cheap content sanity test that a random corruption is overwhelmingly unlikely to pass. Together they ensure only a structurally intact, plausibly genuine payload is ever acted on.

Case 13 — An empty pending bitmap **FIXED**

Problem. A flush is triggered (e.g. timer) when there is actually nothing to send.

Failure if unhandled. A PIGGY TLV with `count==0` would carry a zero bitmap — pure overhead that confirms nothing and wastes a packet.

Mechanism. `module1_get_pending_piggy` returns 0 when `count==0 && !has_queued`, and `build_piggy_tlv` returns NULL on that, so no TLV is constructed.

Why sufficient. The emptiness test gates construction at the single choke point through which all PIGGY builds pass, so no empty TLV can be produced by any trigger path.

Case 14 — Chunk is on disk but unconfirmed on this connection **BY DESIGN**

Problem. The shared store physically contains the chunk, but this particular connection has received no piggyback confirming it.

Failure if unhandled. Treating “in store” as “safe to HIT” would send a HIT to a peer that may not have the chunk in *its* store (a different or fresh peer), which it could not reconstruct.

Mechanism. `piggy_is_confirmed` requires `store_verified==1` and range membership; presence in the store is checked separately and is not sufficient on its own. So the chunk is sent as MISS.

Why sufficient (and intentional). Confirmation is per-connection precisely because the store is shared across peers. Requiring a connection-local proof before HIT is the conservative choice: the only cost is a redundant MISS, whereas the alternative (an unconfirmed HIT) is unrecoverable without NAK.

Case 15 — Two piggybacks arrive out of order **FIXED**

Problem. Network reordering delivers a later window before an earlier one (e.g. the current window before its queued predecessor).

Failure if unhandled. Order-dependent merging could mis-place ranges or overwrite state.

Mechanism. Each PIGGY TLV is fully self-describing (its own epoch+base+bitmap) and `piggy_insert_range` merges by value, not by arrival order — it locates the correct sorted position and fuses overlaps/adjacencies wherever they land.

Why sufficient. Range merging is commutative: inserting [100,163] then [164,200] yields the same [100,200] as the reverse order. Because no step depends on sequence, reordering cannot change the final range set.

Case 16 — Magic bytes appear by chance inside chunk data **FIXED**

Problem. A real chunk's payload happens to contain the bytes `0xAB 0xCD`.

Failure if unhandled. A decoder that *scanned* the byte stream for magic bytes to detect piggybacks would false-trigger on this data and misinterpret chunk bytes as a confirmation.

Mechanism. PIGGY is an ordinary TLV identified solely by its type byte (`0xFC`) and a declared length. The decoder reads exactly that many value bytes; it never scans MISS/HIT data looking for magic. The magic inside a PIGGY payload is only a post-identification sanity check.

Why sufficient. Detection is structural (type + length framing), not pattern-based, so the content of chunk data is never inspected for control meaning. Magic-in-data is therefore just data, with zero chance of being mistaken for a piggyback.

Case 17 — The two sides number chunks differently **FIXED**

Problem. The sender's local cid for a chunk is not the same integer as the receiver's local cid for the same chunk (independent stores assign ids independently).

Failure if unhandled. If the receiver confirmed *its own* cid, the bits would mean different chunks to each side, and the sender would mark the wrong chunks confirmed.

Mechanism. The MISS TLV prepends the *sender's* 4-byte cid. On store, the receiver passes that `sender_cid` (not its local cid) to `piggy_record_confirmed`, so the bitmap is expressed entirely in the sender's id space.

Why sufficient. Both sides now agree on a single id namespace — the sender's — for the bitmap. The receiver's local cid is an internal detail that never crosses the wire in a confirmation, so the two numbering schemes can never disagree about what a bit means.

Case 18 — Confirming before the data is durable **FIXED**

Problem. A chunk is received and TCP-ACKed, but its bytes are still only in the page cache, not on disk; or the disk is full; or the store index is full.

Failure if unhandled. Confirming on receipt would let the sender send HITs for chunks the receiver might lose on a crash before flush, or never stored at all — unrecoverable on the LAN side.

Mechanism. Confirmation is strictly ordered after durability: `piggy_record_confirmed` is called *only* when `store_chunk_verified` returns non-zero, and that function returns non-zero only after a successful `fflush`. Disk-full, flush-failure, and index-full all return 0 → no confirmation.

Why sufficient. The confirmation is causally downstream of the flush, so a bit is set only for data proven on disk. Any failure in the durability path short-circuits to "return 0" before the confirmation call is reached, making over-confirmation structurally impossible.

Case 19 — Reconnect leaves stale pending state **FIXED**

Problem. A connection object is reused for a new connection while a half-built TLV chain or a partially filled pending window remains from the previous one.

Failure if unhandled. Old confirmations or queued mbufs could leak into the new connection, mixing two connections' state.

Mechanism. `module1_state_init` frees any lingering `tlv_pending` chain and `memset`s the whole state to zero (clearing the pending window, ranges, and `store_verified`), then restores only the reverse socket and installs the new epoch.

Why sufficient. A full zeroing leaves no field carrying old data, and the freed mbuf chain prevents a leak. Combined with the new epoch (Case 11), the new connection begins from a provably clean slate.

Case 20 — A HIT lands on a receiver that lacks the chunk **FIXED (via NAK)**

Problem. Despite the per-connection guards, a HIT is sent for a chunk the receiver cannot find (e.g. the store was corrupted or pruned after the confirmation).

Failure if unhandled. The receiver cannot reconstruct the chunk, and the LAN application silently loses data with no recovery path.

Mechanism. The receiver answers a failed HIT lookup with a NAK carrying the hash. The sender re-reads the chunk and resends it as a MISS, and clears its confirmation state (`store_verified=0` , `nranges=0`) so it reverts to MISS-only until a fresh piggyback rebuilds trust.

Why sufficient (with a side effect). NAK turns a potential silent data loss into a self-healing retransmit: delivery resumes via the MISS, and dedup re-enables once new confirmations arrive. The side effect is bluntness — one NAK clears *all* ranges, not just the offending cid — which is safe but temporarily disables dedup for the whole connection (see §11).

11 Known issues & tuning

1. **PIGGY_PAYLOAD_SIZE must be 18, not 14.** The real payload is $\text{magic}(2) + \text{epoch}(4) + \text{base}(4) + \text{bitmap}(8) = 18$ bytes, and the serialiser writes `payload[0..17]` into a stack array declared `uint8_t payload[PIGGY_PAYLOAD_SIZE]`. At 14 that array is 4 bytes too short — a stack overrun on every PIGGY build — and the bitmap occupies bytes 10–17, which the constant must cover. Both ends currently agree only because they share the same code. **Fix: set the constant to 18.** Highest priority.
2. **Single queued-window slot (Case 8).** Out-of-order confirmation across more than ~128 cids forces frequent flushes — correct but chattier. A deeper queue would reduce PIGGY traffic.
3. **NAK clears the whole range list (Case 20).** One spurious NAK reverts the connection to MISS-only. A surgical remove-single-cid helper (noted as future work in the code) would localise it.
4. **MAX_CONFIRMED_RANGES = 16.** Covers 512–1024 in-flight chunks normally. If the “range list full” warning appears, raise it; the sender stays safe but loses dedup on the dropped range until it merges back.

This document covers the piggyback subsystem only. Every behaviour is traced to the code in `module1.c/h` (receiver accumulation, sender range list, all guards) and `module4.c` (the PIGGY TLV build/parse path). “By design” cases are intentional; §11 lists the only places where a constant disagrees with runtime behaviour or where the code itself notes future work.